Warhammer40KApp Study Guide

Purpose: help you explain and defend the code, the React Native architecture, Redux, AsyncStorage, Firebase Auth, Firestore, forms, navigation, and modals.

Project: an Expo React Native app where users log in, create Warhammer 40K army compositions, add/remove units, track points, change theme, and manage profile settings.

Table of Contents

1. App Structure and Startup Flow
2. Authentication Context and Firebase Auth
3. Theme Context and AsyncStorage
4. Navigation: Auth Stack, App Drawer, and Screen Stacks
5. Redux Store, Redux Toolkit, and Persisted State
6. Firebase Services and Firestore Data Flow
7. Forms, Formik, and Yup Validation
8. Main Screens and User Flows
9. Reusable Components: Cards, Buttons, Header, Modal
10. TypeScript Types and Why They Matter
11. Defense Questions and Short Answers

1. App Structure and Startup Flow

Important file: App.tsx

`App.tsx` is the root of the app. It decides which global providers exist and which navigation flow the user sees.

```
export default function App() {
  return (
    <Provider store={store}>
      <PersistGate loading={null} persistor={persistor}>
        <SafeAreaProvider>
          <AuthProvider>
            <ThemeProvider>
              <AppContent />
            </ThemeProvider>
          </AuthProvider>
        </SafeAreaProvider>
      </PersistGate>
    </Provider>
  );
}
```

How this works

- `Provider` comes from `react-redux`. It makes the Redux store available to the entire component tree.
- `PersistGate` waits until persisted Redux data has been loaded from local storage.
- `SafeAreaProvider` gives screens information about safe screen areas, such as notches and status bars.
- `AuthProvider` exposes `currentUser` and `loading` globally.
- `ThemeProvider` exposes `theme` and `toggleTheme` globally.
- `AppContent` can use those contexts because it is inside the providers.

> **Defense answer:** Providers are placed high in the tree because many screens need access to the same global data. Authentication, theme, Redux state, and safe-area information are app-wide concerns, so they belong near the root.

AppContent

```
const AppContent = () => {
  const { currentUser, loading } = useAuth();
  const [fontsLoaded] = useLoadFonts();
  const [showSplashScreen, setShowSplashScreen] = useState(true);

  useLoadArmyCompositions();
```

```
  useEffect(() => {
    const timer = setTimeout(() => {
      setShowSplashScreen(false);
    }, 2000);

    return () => clearTimeout(timer);
  }, []);

  if (loading || showSplashScreen || !fontsLoaded) {
    return <SplashScreen />;
  }

  return (
    <NavigationContainer>
      {currentUser ? <BattleForgeInformationDrawerStack /> : <AuthStack />}
    </NavigationContainer>
  );
};
```

What each part means

- `useAuth()` reads the authentication context.
- `useLoadFonts()` loads the custom font before showing normal UI.
- `showSplashScreen` forces the splash screen to remain visible for about two seconds.
- `useLoadArmyCompositions()` starts syncing the user's Firestore army compositions into Redux when logged in.
- The `if` statement prevents the app from showing the wrong screen while auth or fonts are still loading.
- `NavigationContainer` is required by React Navigation. It manages navigation state.

> **Defense answer:** The app waits for authentication, fonts, and the splash delay because showing the real app too early could cause flicker or briefly show the wrong navigation stack.

2. Authentication Context and Firebase Auth

Important file: src/contexts/AuthContext.tsx

This file creates a global authentication state. It lets every screen know whether a user is logged in.

```
type AuthContextType = {
  currentUser: User | null;
  loading: boolean;
};
```

`currentUser` can be a Firebase `User` object or `null`. It is `null` when nobody is logged in. `loading` tells the app whether Firebase is still checking the saved login session.

```
export const AuthContext = createContext<AuthContextType>({
  currentUser: null,
  loading: true,
});
```

`createContext` creates a shared data container. The object passed to it is the default value. The real values come from `AuthProvider`.

The provider

```
export const AuthProvider = ({ children }: AuthProviderProps) => {
  const [currentUser, setCurrentUser] = useState<User | null>(null);
  const [loading, setLoading] = useState(true);

  useEffect(() => {
    const unsubscribe = onAuthStateChanged(auth, (user) => {
      setCurrentUser(user);
      setLoading(false);
    });

    return unsubscribe;
  }, []);

  return (
    <AuthContext.Provider value={{ currentUser, loading }}>
      {children}
    </AuthContext.Provider>
  );
};
```

How Firebase Auth works here

- `onAuthStateChanged` starts listening to Firebase Auth.

- Firebase calls the callback when the user logs in, logs out, or when the app reloads and Firebase checks saved login data.
- The `user` parameter is either a Firebase user object or `null`.
- `setCurrentUser(user)` updates React state.
- `setLoading(false)` tells the app that the Firebase check is finished.

Why return unsubscribe?

`onAuthStateChanged` subscribes to auth changes and returns a function that stops the subscription. React calls a returned function from `useEffect` as cleanup.

```
return unsubscribe;
```

This is the same idea as:

```
return () => {
  unsubscribe();
};
```

> **Defense answer:** I used Context for auth because login state is needed globally. The listener keeps the app synchronized with Firebase, and the cleanup prevents the listener from staying active after the provider is removed.

useAuth hook

```
export const useAuth = () => {
  return useContext(AuthContext);
};
```

This hook is a small wrapper around `useContext`. Instead of importing `useContext` and `AuthContext` everywhere, screens can simply call `useAuth()`.

3. Theme Context and AsyncStorage

Important file: src/contexts/ThemeContext.tsx

This file manages the light/dark theme globally.

```
type Theme = "light" | "dark";
```

This TypeScript union type means the theme can only be exactly `"light"` or `"dark"`. This prevents accidental values like `"blue"` or `"Dark"`.

```
type ThemeContextType = {
  theme: Theme;
  toggleTheme: () => void;
};
```

The theme context exposes two things: the current theme value and a function to change it.

Loading saved theme

```
useEffect(() => {
  const loadTheme = async () => {
    const savedTheme = await AsyncStorage.getItem("theme");

    if (savedTheme === "light" || savedTheme === "dark") {
      setTheme(savedTheme);
    }
  };

  loadTheme();
}, []);
```

`AsyncStorage` is local key-value storage on the device. It is asynchronous, so the code uses `async` and `await`.

- `AsyncStorage.getItem("theme")` reads the saved value.
- The `if` check protects the app from invalid stored values.
- `setTheme(savedTheme)` updates the UI theme.
- The empty dependency array means this runs once when the provider mounts.

Toggling theme

```
const toggleTheme = async () => {
  const newTheme = theme === "dark" ? "light" : "dark";

  setTheme(newTheme);
```

```
  await AsyncStorage.setItem("theme", newTheme);
};
```

The ternary operator means: if the current theme is dark, switch to light; otherwise switch to dark. The UI updates immediately with `setTheme`, then the new choice is saved locally.

> **Defense answer:** I used AsyncStorage for the theme because it is a local user preference. It does not need to be stored in Firestore because it is not shared army data.

4. Navigation: Auth Stack, App Drawer, and Screen Stacks

React Navigation concept

React Navigation controls which screen is visible. This project uses stack navigators and a drawer navigator.

- A **stack navigator** is like a pile of pages. You navigate forward, and you can go back.
- A **drawer navigator** is a side menu for switching between larger sections of the app.

AuthStack

```
export type AuthStackParamList = {
  Login: undefined;
  Register: undefined;
};
```

This type says the auth stack has two screens. `undefined` means those screens do not require route parameters.

```
<Stack.Navigator screenOptions={{ headerShown: false }}>
  <Stack.Screen name="Login" component={LoginScreen} />
  <Stack.Screen name="Register" component={RegisterScreen} />
</Stack.Navigator>
```

The header is hidden because login and register screens use their own layout.

BattleForgeStack

```
export type BattleForgeStackParamList = {
  Home: undefined;
  CreateArmyComposition: undefined;
  ArmyComposition: { armyComposition: ArmyComposition };
};
```

`ArmyComposition` needs data about the selected composition, so it receives an `armyComposition` route parameter.

Drawer stack

```
<Drawer.Screen
  name="BattleForgeStack"
  component={BattleForgeStack}
```

```
  options={{ title: "Battle Forge" }}
/>
```

The drawer does not show individual screens like Home and Profile. Instead, it shows big sections. Each drawer item contains its own stack navigator.

> **Defense answer:** I use stacks for flows, such as Home to Create Army to Army details. I use a drawer for top-level sections, such as Battle Forge and Information.

5. Redux Store, Redux Toolkit, and Persisted State

Important files

- `src/store/index.ts`
- `src/store/hooks.ts`
- `src/features/armyCompositions/armyCompositionSlice.ts`

What Redux does in this app

Redux stores the list of army compositions in one shared place. Screens can read from it without passing props through many layers.

Firestore is the remote database. Redux is the local app state. They work together.

Store setup

```
const rootReducer = combineReducers({
  armyCompositions: armyCompositionsReducer,
});
```

This means Redux state has a property called `armyCompositions`, controlled by `armyCompositionsReducer`.

```
const persistedReducer = persistReducer(persistConfig, rootReducer);
```

`redux-persist` wraps the reducer so Redux state can be saved to `AsyncStorage`.

Slice concept

```
export const armyCompositionSlice = createSlice({
  name: "armyCompositions",
  initialState,
  reducers: {
    armyCompositionAdded(state, action: { payload: ArmyComposition }) {
      state.push(action.payload);
    },
  },
});
```

A slice contains the state and the reducer functions for one feature. Redux Toolkit uses Immer internally, so code like `state.push` is allowed even though Redux state must stay immutable.

Important reducers

- `armyCompositionsLoaded`: replaces local Redux data with data loaded from Firestore.

- `armyCompositionsCleared`: empties the list when logging out.
- `armyCompositionAdded`: adds a new composition to Redux.
- `armyCompositionUnitsUpdated`: updates the units and total points of one composition.
- `armyCompositionDeleted`: removes a composition by id.

Typed Redux hooks

```
export const useAppDispatch = useDispatch.withTypes<AppDispatch>();
export const useAppSelector = useSelector.withTypes<RootState>();
```

These hooks make Redux usage type-safe. TypeScript knows what actions can be dispatched and what the state shape looks like.

> **Defense answer:** Redux is useful here because army compositions are shared between multiple screens. Redux Toolkit reduces boilerplate and makes reducers easier to write safely.

6. Firebase Services and Firestore Data Flow

Firebase config

```
const app = initializeApp(firebaseConfig);

export const auth = getAuth(app);
export const database = getFirestore(app);
```

`initializeApp` connects the app to your Firebase project. `auth` is used for login/register/logout. `database` is the Firestore database connection.

Auth service

```
export const login = (auth: Auth, email: string, password: string) => {
  return signInWithEmailAndPassword(auth, email, password);
};
```

The service function hides Firebase details from the screen. The screen calls `login`, and Firebase handles credential checking.

Firestore structure

```
users/{userId}/armyCompositions/{armyCompositionId}
```

This means each user has their own subcollection of army compositions. This is important because one user's armies should not appear for another user.

Live Firestore subscription

```
return onSnapshot(
  collection(database, "users", userId, "armyCompositions"),
  (querySnapshot) => {
    const armyCompositions = querySnapshot.docs.map((document) => {
      return document.data() as ArmyComposition;
    });

    onArmyCompositionsChanged(armyCompositions);
  },
);
```

`onSnapshot` listens in real time. When Firestore data changes, Firebase sends a new snapshot. The app converts Firestore documents into normal TypeScript objects and sends them back through the callback.

Saving data

```
return setDoc(
  doc(database, "users", userId, "armyCompositions", armyComposition.id),
  armyComposition,
);
```

`doc` creates a reference to one Firestore document. `setDoc` writes data to that document.

Updating only units and total points

```
return setDoc(
  doc(database, "users", userId, "armyCompositions", armyCompositionId),
  { units, totalPoints },
  { merge: true },
);
```

`merge: true` means Firestore updates only the provided fields. It keeps other fields like `name` and `army`.

> **Defense answer:** I put Firebase calls in service files to keep screens focused on UI. Services make the app easier to test, read, and change.

7. Forms, Formik, and Yup Validation

Why Formik?

Forms need to track input values, touched fields, validation errors, and submit handling. Formik manages that state so the component does not need many separate `useState` calls.

LoginForm example

```
<Formik
  initialValues={{ email: "", password: "" }}
  validationSchema={loginValidationSchema}
  onSubmit={(values) => {
    handleLogin(values.email, values.password);
  }}
>
```

- `initialValues` defines the starting form values.
- `validationSchema` connects Yup validation rules.
- `onSubmit` runs after validation succeeds.

TextInput controlled by Formik

```
<TextInput
  value={props.values.email}
  onChangeText={props.handleChange("email")}
  onBlur={props.handleBlur("email")}
/>
```

The input value comes from Formik. When the user types, Formik updates the value. When the input loses focus, Formik marks the field as touched.

Yup validation

```
export const registerValidationSchema = Yup.object({
  email: Yup.string()
    .email("Enter a valid email")
    .required("Email is required"),

  password: Yup.string()
    .min(6, "Password must be at least 6 characters")
    .required("Password is required"),

  confirmPassword: Yup.string()
    .oneOf([Yup.ref("password")], "Passwods must match")
```

```
    .required("Confirm password is required"),
});
```

`Yup.ref("password")` means the confirm password must match the password field.

> **Small note:** The message says `Passwods`. That is only a spelling issue in the validation message, not a logic problem.

> **Defense answer:** Formik manages form state, and Yup keeps validation rules separate and reusable. This makes the form components cleaner.

8. Main Screens and User Flows

LoginScreen and RegisterScreen

These screens show forms and call Firebase service functions.

```
const handleLogin = async (email: string, password: string) => {
  try {
    setFirebaseError("");
    await login(auth, email, password);
  } catch {
    setFirebaseError("Login failed");
  }
};
```

If login succeeds, Firebase Auth updates. Then `AuthContext` receives the new user through `onAuthStateChanged`. The app automatically switches from `AuthStack` to the main drawer.

HomeScreen

```
const { armyCompositions, deleteComposition } = useArmyCompositions();
```

The screen does not directly use Redux or Firestore. The custom hook gives it exactly what it needs: the list and a delete function.

CreateArmyCompositionScreen

This screen loads available armies from Firestore and lets the user choose one. The form collects the army composition name.

```
const {
  selectedArmy,
  armySelectionError,
  selectArmy,
  createArmyComposition,
} = useCreateArmyComposition(navigation);
```

The custom hook owns creation logic. The screen focuses on displaying army options and the form.

ArmyCompositionScreen

```
const { armyComposition } = route.params;
```

The selected army composition is passed through navigation parameters from the previous screen.

```
const { selectedUnits, totalPoints, isOverPointLimit, addUnit, removeUnit } =
  useArmyCompositionUnits(armyComposition);
```

This hook manages unit selection, point calculation, Redux updates, and Firestore updates.

ProfileScreen

This screen uses auth, theme, Redux, and Firebase services.

```
const handleLogout = async () => {
  dispatch(armyCompositionsCleared());
  await logout(auth);
};
```

It clears local Redux army data before signing out. This prevents another user from seeing the previous user's local army list.

> **Defense answer:** The screens mostly compose UI and call hooks. Complex behavior is moved into hooks and services, so screens stay readable.

9. Reusable Components: Cards, Buttons, Header, Modal

Component props

Props are values passed from a parent component to a child component. They make components reusable.

ArmyCompositionCard

```
type ArmyCompositionCardProps = {
  armyComposition: ArmyComposition;
  onPress: () => void;
  onDelete: () => void;
};
```

- `armyComposition` is the data to display.
- `onPress` is called when the user opens the card.
- `onDelete` is called when the user presses the trash icon.

UnitCard

```
type UnitCardProps = {
  unit: Unit;
  buttonText: string;
  onPress: () => void;
};
```

The same card can be used in two situations. In the modal, `buttonText` is `"+"`. In the selected unit list, it is `"-"`. This avoids duplicate components.

Modal

```
<Modal visible={visible} animationType="slide">
```

A React Native `Modal` renders content above the current screen. In this app, `SelectUnitModal` lets the user choose a unit without leaving the army composition screen.

```
type SelectUnitModalProps = {
  visible: boolean;
  units: Unit[];
  totalPoints: number;
  maxPoints: number;
  onClose: () => void;
```

```
  onAddUnit: (unit: Unit) => void;
};
```

- `visible` controls whether the modal is shown.
- `units` is the list of valid units for the selected army.
- `totalPoints` and `maxPoints` show the point status.
- `onClose` closes the modal.
- `onAddUnit` sends the chosen unit back to the screen.

> **Defense answer:** Reusable components reduce duplication. Props let each parent decide what data the component shows and what should happen when the user interacts with it.

10. TypeScript Types and Why They Matter

Army

```
export type Army = {
  id: string;
  name: string;
  armyRule: string;
  imageKey: ArmyImageKey;
  units: Unit[];
};
```

An army has an id, name, rule, image key, and list of units. The image key links Firestore data to local image assets.

Unit

```
export type Unit = {
  id: string;
  name: string;
  movement: number;
  toughness: number;
  save: number;
  wounds: number;
  leadership: number;
  objectControl: number;
  ability: string;
  points: number;
};
```

This type describes the game stats for a unit.

ArmyComposition

```
export type ArmyComposition = {
  id: string;
  name: string;
  army: Army;
  units: SelectedUnit[];
  totalPoints: number;
};
```

An army composition is the user's saved list. It contains the selected army, selected units, and total points.

Why TypeScript helps

- It catches wrong property names before runtime.
- It documents the shape of the data.
- It makes navigation parameters safer.
- It improves autocomplete in the editor.

> **Defense answer:** TypeScript gives the project stronger safety. It helps prevent common bugs, especially when passing data between screens, Redux, Firebase, and components.

11. Defense Questions and Short Answers

Why use Context for auth and theme?

Because both are global app state. Many screens need them, and passing them manually through props would be messy.

Why use Redux if you already use Context?

Context is good for small global values like auth and theme. Redux is better for feature state with multiple actions, such as adding, updating, loading, and deleting army compositions.

Why use AsyncStorage?

AsyncStorage saves small local data on the device. This app uses it for theme preference and Redux persistence.

Why use Firestore?

Firestore stores user army compositions remotely, so data is saved beyond one app session and can be synced from the backend.

Why use onSnapshot instead of getDocs for army compositions?

`getDocs` loads data once. `onSnapshot` keeps listening and updates the app when Firestore data changes.

Why use getDocs for armies?

The selectable armies are shared reference data. They only need to be loaded when the create screen needs them, so a one-time fetch is enough.

Why use Formik and Yup?

Formik manages form values and submit behavior. Yup defines validation rules clearly and separately.

Why give selected units a separate armyCompositionUnitId?

The same unit type can be added more than once. A separate selected-unit id lets the app remove one specific copy instead of all copies of the same unit.

Why clear Redux data on logout?

To prevent the next logged-in user from seeing local data from the previous user before Firestore finishes loading.

What is a hook?

A hook is a function that lets a component use React features such as state, context, and effects. Custom hooks in this project package reusable logic.

What is useEffect?

`useEffect` runs side effects, such as starting Firebase listeners, loading data, reading AsyncStorage, or setting timers. If it returns a function, React uses that function as cleanup.

What is state?

State is data that can change and cause the UI to re-render. Examples are `currentUser`, `loading`, `theme`, `selectedArmy`, and `modalVisible`.

What is the difference between props and state?

Props are passed into a component by its parent. State belongs to the component or hook and can change over time.

What is a reducer?

A reducer is a function that receives the current state and an action, then returns the next state. Redux Toolkit organizes reducers inside slices.

What is a modal?

A modal is a temporary UI layer displayed above the current screen. This app uses it to let the user choose a unit while staying inside the army composition flow.